

Zynq-7000 · mainline U-Boot · Yocto · kexec · Yosys / nextpnr · REST

An Open Bitstream Pipeline for the ALINX AX7020

From a blank board to a REST endpoint that accepts a bitstream and loads it into the FPGA: JTAG bring-up with mainline U-Boot, a self-updating Yocto Linux in QSPI flash, an open-toolchain bitstream, and a systemd-managed API. No SD card, no serial cable, no vendor tool in the loop.

Stephan Bökelmann

MaxClerkwell · Bochum, Germany · nabla B

 maxclerkwell.tech

 [@MaxClerkwell](https://twitter.com/MaxClerkwell)

 [@MaxClerkwell](https://www.youtube.com/@MaxClerkwell)

 [Full Stack Engineering](#)

 [maxclerkwell](https://github.com/maxclerkwell)

 [accelerator-stephan](https://www.linkedin.com/company/accelerator-stephan)

 [@_maxclerkwell](https://www.instagram.com/_maxclerkwell)

 stephan@boekelmann.net

Companion document to the five-part article series at maxclerkwell.tech/alinx. All sources, recipes and scripts: github.com/MaxClerkwell/ax7020-bringup.

The board was provided by [ALINX](#); the vendor had no influence on the content.

Version 1.0

15 September 2026

Board: AX7020, XC7Z020-2CLG400

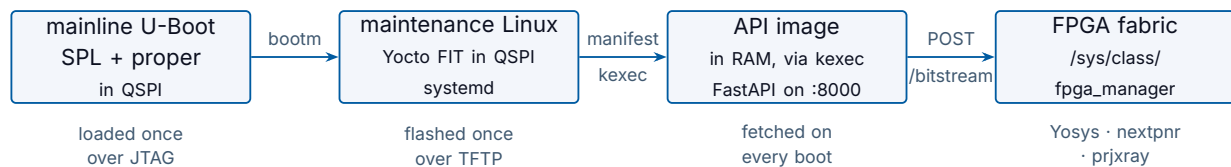
Contents

1	What this tutorial builds	3
1.1	Scope	3
1.2	What you need	3
1.3	Board facts used throughout	4
1.4	Repository	4
2	Host setup	4
2.1	Toolchain and tools	4
2.2	JTAG adapter	4
2.3	Lab network	5
2.4	Console and TFTP	5
3	Stage 1: Mainline U-Boot in QSPI flash	5
3.1	The four files	5
3.2	Build	6
3.3	Flash layout	6
3.4	Bring-up over JTAG	6
3.5	Flash	7
3.6	Standalone boot	7
4	Stage 3: A Yocto Linux in flash	7
4.1	Layers	7
4.2	Build	8
4.3	Boot from RAM first	8
4.4	Flash the FIT	9
5	Stage 2: The image server and the updater	9
5.1	Server	9
5.2	Publishing	10
5.3	The updater	10
6	Stage 4: A bitstream without Vivado	10
6.1	Toolchain	11
6.2	Design	11
6.3	Flow	11
6.4	Load over SSH	12
7	Stage 5: The bitstream API as an image	12
7.1	The service	12
7.2	Recipes	12
7.3	Build, publish, and update the maintenance system	13
7.4	The cold boot	13
8	Operating the board	14
8.1	Finding it	14

- 8.2 SSH 14
 - 8.3 Rolling back an image 14
 - 8.4 Rewriting flash 14
 - 8.5 Backing up flash 14
- 9 Where to go from here 15
- About the author 16

1 What this tutorial builds

The ALINX AX7020 is a Zynq-7000 development board: two Cortex-A9 cores, a 7-series FPGA fabric on the same die, 1 GiB of DDR3, 32 MiB of QSPI flash and Gigabit Ethernet on the processor side. The vendor path to a running system goes through Vivado, the Xilinx FSBL, PetaLinux and an SD card. This tutorial takes the other path and replaces every one of those with a component whose source you can read.



When you are done, this is what a power cycle looks like, with nothing attached but power and Ethernet:

```

U-Boot: DHCP client bound to address 10.42.100.153
U-Boot: ## Loading kernel (any) from FIT Image at 02000000 ...
maintenance system -> ax7020-update.service -> image server -> kexec
API image: GET http://10.42.100.156:8000/state -> {"state":"unknown", ...}
POST blinky.bit -> {"state":"operating", "idcode":"0x03727093", ...}
  
```

1.1 Scope

This is the clean path: every command that is needed, in the order it is needed, with a checkpoint after each step. The reasons behind the design decisions, and the considerable number of ways each step can go wrong, are in the article series at maxclerkwell.tech/alinx. Where a fact from that series is *required* to get through a step, it appears here as a rule; where it is merely interesting, it does not.

1.2 What you need

- An ALINX AX7020 with its 12 V supply and an Ethernet cable.
- A JTAG adapter. An FTDI FT232H breakout is enough; no reset or buffer-enable line is needed on this board.
- A Linux workstation (Debian 13 was used) with a second Ethernet port or USB NIC for a private lab network, roughly 100 GB of free disk for Yocto, and a few hours of build time.
- A machine on the company or lab network that can run one nginx container. A router or NAS is ideal.
- No SD card, no USB-UART cable, no Vivado, no PetaLinux.

1.3 Board facts used throughout

SoC	XC7Z020-2CLG400, JTAG IDCODE 0x23727093
DDR3	2 × MT41J256M16, 32-bit bus, 533 MHz, 1GiB
QSPI	Winbond W25Q256, 32 MiB, single chip select, MIO 1..6
Ethernet	PS GEM0, RGMII on MIO 16..27, RTL8211E-VL PHY at address 1
Boot jumper	J13 : left = SD, middle = QSPI, right = JTAG
PL clock	50 MHz oscillator on pin U18
PL LEDs	M14, M15, K16, J16, active high, LVCMOS33

1.4 Repository

Every file referenced below is in github.com/MaxClerkwell/ax7020-bringup. Paths in this document are relative to its root.

uboot/	DTS, defconfig, ps7_init for the mainline U-Boot tree
openocd/	adapter definition and the JTAG bring-up script
tools/	netconsole shell, TFTP server, publish and flash scripts
yocto/	meta-ax7020 layer and the build setup script
bitstream/	open toolchain flow, blinky and empty designs, bit2bin.py
api/	the FastAPI service
docs/	session walkthroughs (German), longer than this tutorial

2 Host setup

2.1 Toolchain and tools

```
sudo apt install gcc-arm-none-eabi openocd python3 make git
sudo apt install gawk wget diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3-pip python3-pexpect xz-utils debianutils \
    iputils-ping python3-git python3-jinja2 python3-subunit zstd lz4 file \
    locales libacl1
```

The second line is Yocto's host dependency list for Debian. Note `lz4` rather than the older `liblz4-tool`; Yocto still looks for the binary under its old name `lz4c`, and [yocto/hostbin/lz4c](#) is a symlink that covers this without root.

2.2 JTAG adapter

[openocd/ft232h.cfg](#) describes a bare FT232H. On FTDI chips the MPSSE pin assignment is fixed in silicon (ADBUS0 = TCK, 1 = TDI, 2 = TDO, 3 = TMS), so only idle levels and directions are declared:

```
adapter driver ftdi
ftdi vid_pid 0x0403 0x6014
ftdi layout_init 0x0008 0x000b
adapter speed 1000
transport select jtag
```

```
openocd -f openocd/ft232h.cfg -f target/zynq_7000.cfg
```

✓ **Checkpoint** The chain shows two TAPs and both cores:

```
JTAG tap: zynq_pl.bs tap/device found: 0x23727093 (Xilinx, part 0x3727)
JTAG tap: zynq.cpu tap/device found: 0x4ba00477 (ARM Ltd)
zynq.cpu0: hardware has 6 breakpoints, 4 watchpoints
zynq.cpu1: hardware has 6 breakpoints, 4 watchpoints
```

2.3 Lab network

For bring-up the board hangs on a private network behind the workstation. A NetworkManager *shared* connection provides DHCP and keeps the board off the house network:

```
nmcli connection add type ethernet ifname <iface> con-name zynq-lab \
    ipv4.method shared ipv4.addresses 192.168.77.1/24 ipv6.method disabled
nmcli connection up zynq-lab
```

Find the board's address with `ip neigh show dev <iface>` after it has taken a lease.

2.4 Console and TFTP

There is no serial cable in this setup. U-Boot's console is its *netconsole* over UDP port 6666, and [tools/ncsh.py](#) is the shell for it. [tools/tftpd.py](#) is an unprivileged TFTP server, so no system service is needed:

```
tools/tftpd.py tftp 6969 & # serves the tftp/ directory
tools/ncsh.py <board-ip> # interactive
tools/ncsh.py <board-ip> "bdinfo" "sf probe 0 30000000 0" # one-shot
```

On the board, `setenv tftpdstp 6969` points tftpboot at that port.

3 Stage 1: Mainline U-Boot in QSPI flash

Mainline U-Boot has no AX7020 support. Four files teach it the board; they are mirrored in [uboot/](#) and go into a checkout of U-Boot v2026.10-rc2 or later.

3.1 The four files

[uboot/zynq-ax7020.dts](#)

[arch/arm/dts/](#). Board description, every value derived from the vendor's hardware handoff and schematic.

[uboot/alinx_ax7020_defconfig](#)

[configs/](#). Derived from `xilinx_zynq_virt_defconfig`: netconsole, legacy network stack, TFTP put, md5sum; SPL FIT loading and the relocated SPL stack switched off.

[uboot/ps7_init_gpl.c, .h](#)

[board/xilinx/zynq/zynq-ax7020/](#). The PS initialisation Vivado generates (PLLs, MIO, DDR timing), extracted from the vendor's XSA archive, which is a plain zip. Five declarations

gained (void) for GCC 15.

Two edits complete it: add `zynq-ax7020.dtb` to `arch/arm/dts/Makefile`, and give the default environment in `include/configs/zynq-common.h` `ncip = 255.255.255.255` and `stdin/stdout/stderr = serial,nc`, so the netconsole is armed before the environment is ever writable.

⚠ Rule Two device-tree facts are mandatory on this board. `bootph-all` must be on the flash node *and* the QSPI controller, or SPL finds no chip select. And `spi-rx-bus-width` must be `<1>`: quad read returns data three bytes off inside SPL, and U-Boot proper will not show the fault.

3.2 Build

The boot command is compiled in rather than stored in the environment, because `saveenv` does not work in JTAG boot mode and long `setenv` lines break over the netconsole. It reads 24 MiB from the FIT partition:

```
CONFIG_BOOTCOMMAND="sf probe 0 30000000 0; sf read 0x20000000 0x340000 0x1800000; bootm
0x20000000"
```

```
cd u-boot
make alinx_ax7020_defconfig
scripts/config --disable TOOLS_MKEFICAPSULE      # host tool needs gnutls
make olddefconfig
make -j$(nproc) CROSS_COMPILE=arm-none-eabi-
cp spl/boot.bin u-boot.img ../tftp/
```

✓ **Checkpoint** `spl/boot.bin` (about 107 kB, SPL with the Xilinx boot header) and `u-boot.img` (about 1.1 MB, legacy ulmage) exist.

3.3 Flash layout

Offset	Content	Slot	mtd
0x000000	<code>boot.bin</code>	1 MiB	0
0x100000	<code>u-boot.img</code>	2 MiB	1
0x300000	environment + redundant copy	256 KiB	2
0x340000	FIT: kernel + DTB + initramfs	28.75 MiB	3

The same map is in the Linux device tree, so the running Linux sees the partitions as `/dev/mtd0..3`.

3.4 Bring-up over JTAG

Board cold, J13 on JTAG, power on. `openocd/load-uboot.cfg` loads SPL into on-chip memory, waits for `ps7_init` to bring up DDR, writes and reads a DDR test word, loads U-Boot proper to `0x4000000`, verifies it and jumps:

```
openocd -f openocd/ft232h.cfg -f target/zynq_7000.cfg -f openocd/load-uboot.cfg
```

⚠ Rule Load U-Boot proper only while *SPL* is halted, with caches off. Loading into DDR while a previous U-Boot proper is halted corrupts the image in 32-byte chunks through its enabled D-cache. And after any QSPI-mode boot, a JTAG-mode power cycle is mandatory before loading over JTAG again: `ps7_init` is not idempotent.

✓ **Checkpoint** Over the netconsole, `bdinfo` reports 1GiB of DRAM and `sf probe 0 30000000 0` detects `w25q256` with 32 MiB.

3.5 Flash

Only from a U-Boot that was JTAG-loaded onto a freshly power-cycled board in JTAG mode. Sizes are the file sizes in hex (`printf "%x\n" $(stat -c %s tftp/boot.bin)`):

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "sf probe 0 30000000 0" \
  "tftpboot 0x10000000 boot.bin" "tftpboot 0x11000000 u-boot.img" \
  "md5sum 0x10000000 <size_boot_bin>" "md5sum 0x11000000 <size_u_boot_img>"
tools/ncsh.py <ip> "sf erase 0 0x300000" \
  "sf write 0x10000000 0 <size_boot_bin>" \
  "sf write 0x11000000 0x100000 <size_u_boot_img>"
tools/ncsh.py <ip> "sf read 0x12000000 0 <size_boot_bin>" \
  "md5sum 0x12000000 <size_boot_bin>" \
  "sf read 0x13000000 0x100000 <size_u_boot_img>" \
  "md5sum 0x13000000 <size_u_boot_img>"
```

✓ **Checkpoint** Both read-back MD5s equal the host's `md5sum` of the files. Nothing counts as flashed until they do.

3.6 Standalone boot

Power off, J13 to QSPI, JTAG unplugged, power on.

✓ **Checkpoint** U-Boot appears on the netconsole by itself, with `modeboot = qspiboot`. The JTAG adapter goes into the drawer; everything from here on happens over Ethernet.

4 Stage 3: A Yocto Linux in flash

Stages are numbered as in the article series, where the image server of Stage 2 ended up being built after the Linux of Stage 3.

Everything Linux needs ships as one artefact: a FIT image holding kernel, device tree and an `initramfs`, written into the FIT partition. The root filesystem lives in RAM. This image is the *maintenance system*: small, rarely changed, and the thing that fetches everything else.

4.1 Layers

`scarthgap` (Yocto 5.0 LTS) throughout, as the newest branch `meta-xilinx` supports:


```
cd yocto
for r in "https://git.yoctoproject.org/poky poky" \
        "https://github.com/Xilinx/meta-xilinx meta-xilinx" \
        "https://git.yoctoproject.org/meta-arm meta-arm" \
        "https://github.com/openembedded/meta-openembedded meta-openembedded"; do
    set -- $r; git clone --depth 1 -b scarthgap "$1" "$2"
done
```

[yocto/meta-ax7020](#) is the board layer. Its parts, paths relative to it:

[conf/machine/ax7020.conf](#)

requires meta-xilinx's zynq-generic, sets `KERNEL_IMAGETYPE` to `fitImage`, declares no bootloader provider, and names `ax7020-initramfs` as the initramfs that goes into the FIT.

[recipes-kernel/linux/files/zynq-ax7020.dts](#)

the Linux device tree, including the flash partition map.

[recipes-kernel/linux/files/ax7020.cfg](#)

kernel fragment: Zynq FPGA manager, FPGA regions and overlays, netconsole, MTD and QSPI, kexec; graphics, sound, wireless and IPv6 off.

[recipes-core/images/ax7020-initramfs.bb](#)

the maintenance image: core-image-minimal as `cpio.gz` plus dropbear, curl, kexec-tools, the updater and the operator's SSH key.

[recipes-support/ax7020-ssh-key](#)

installs `authorized_keys` to `${ROOT_HOME}`. Put your public key in [yocto/meta-ax7020/recipes-support/ax7020-ssh-key/files/authorized_keys](#) before building.

[yocto/setup-build.sh](#)

regenerates [build/conf](#) from version control: machine `ax7020`, `INIT_MANAGER = "systemd"`, no bootloader from Yocto.

4.2 Build

```
source yocto/setup-build.sh
setsid nohup bitbake virtual/kernel > build.log 2>&1 &      # detached: a killed
tail -f build.log                                           # client kills the build
```

The first build takes several hours. The FIT lands in [yocto/build/tmp/deploy/images/ax7020/](#) as `fitImage-ax7020-initramfs-ax7020-ax7020`; copy it to [tftp/fitImage](#).

✔ **Checkpoint** `dumpimage -l tftp/fitImage` lists one configuration with a kernel, the DTB `zynq-ax7020.dtb` and a ramdisk, each with a SHA256 hash. The FIT is under 24 MiB.

4.3 Boot from RAM first

From a JTAG-loaded U-Boot, before anything is written:

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "tftpboot 0x2000000 fitImage" "bootm
0x2000000"
```

✔ **Checkpoint** Linux boots, takes a DHCP lease under hostname ax7020, and **port 22 answers** with key-only login. Ping is not the test; a U-Boot with a working network stack also answers ping.

4.4 Flash the FIT

```
tools/ncsh.py <ip> "setenv tftpdstp 6969" "tftpboot 0x2000000 fitImage" \
"sf probe 0 30000000 0" "sf erase 0x340000 0x1CC0000" \
"sf write 0x2000000 0x340000 <size_fit>" \
"sf read 0x3000000 0x340000 <size_fit>" "md5sum 0x3000000 <size_fit>" \
"md 0x3000000 4" "md 0x3800000 4"
```

✔ **Checkpoint** The read-back MD5 matches, *and* the words at two different offsets of the read-back differ from each other. A flash whose address lines are shifted can return a self-consistent, wrong image that passes a single checksum.

Power off, J13 on QSPI, power on.

✔ **Checkpoint** Linux boots from flash on its own and is reachable over SSH. At this point the board can be moved from the lab network to the company network; it will announce itself as ax7020 to DHCP.

5 Stage 2: The image server and the updater

The maintenance system never changes what is in flash. On every boot it asks one URL for a manifest, downloads kernel, device tree and initramfs, verifies them and kexecs into the result. A broken image costs a power cycle, not a reflash, because after the power cycle the maintenance system is back.

5.1 Server

One nginx container on a machine the board can reach, read-only, bound to the LAN address only, serving a directory:

```
docker run -d --name ax7020-image-server --restart unless-stopped \
-p 10.42.0.1:8080:80 \
-v /srv/ax7020-images:/usr/share/nginx/html:ro \
-v /etc/ax7020-image-server/default.conf:/etc/nginx/conf.d/default.conf:ro \
--read-only --tmpfs /var/cache/nginx --tmpfs /var/run --tmpfs /tmp nginx:alpine
```

`default.conf` contains `autoindex on`; and `disable_symlinks off`; . Every build gets a time-stamped directory; `ax7020-latest` is a symlink to the newest one. The board knows exactly one URL:

```
http://10.42.0.1:8080/ax7020-latest/manifest
```

5.2 Publishing

`tools/publish-image.sh` copies `zImage`, `DTB` and `cpio.gz` from the Yocto deploy directory over `scp`, writes the manifest with SHA256 sums, and repoints the symlink:

```
tools/publish-image.sh AI-heimdall:/srv/ax7020-images # API image
tools/publish-image.sh -i ax7020-initramfs AI-heimdall:/srv/ax7020-images #
maintenance image
```

```
kernel zImage          542ef58f...
dtb     zynq-ax7020.dtb 1e3fa9c7...
initrd  initramfs.cpio.gz 4e73ba34...
```

⚠ Rule The manifest names three files, not a FIT. `kexec-tools` on 32-bit ARM has no FIT loader and its `zImage` probe accepts any file; handed a FIT, it jumps into the device tree and the board dies silently. The updater checks the `zImage` magic at offset `0x24` and the `DTB` magic itself, because nothing downstream will.

5.3 The updater

The updater lives in `yocto/meta-ax7020/recipes-support/ax7020-updater/files/`: `ax7020-update` is the shell script, `ax7020-update.conf` sets `IMAGE_URL` (empty means *stay in the maintenance system*), and `ax7020-update.service` runs it once per boot:

```
[Unit]
After=network-online.target time-sync.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStartPre=/bin/sleep 15
ExecStart=/usr/bin/ax7020-update
StandardOutput=journal+console
```

The script fetches the manifest, then each file, compares every SHA256, checks both magics, and runs `kexec -l zImage --dtb --initrd --command-line` followed by `kexec -e`. `DRY_RUN=1` stops before the jump. If anything fails the unit fails and the board stays reachable over SSH.

✓ **Checkpoint** Publish the maintenance image itself, set `IMAGE_URL`, reboot: about twenty seconds later the same system is back with an uptime of seconds, and `journalctl -u ax7020-update` shows three checksums verified.

6 Stage 4: A bitstream without Vivado

6.1 Toolchain

The Zynq-7020 is a 7-series device and is covered by the openXC7 flow. Everything except Yosys is built from source under [build/](#):

Tool	Source	Role
Yosys 0.66	Debian package	synthesis, synth_xilinx
nextpnr-xilinx	openXC7 fork	placement and routing, reads XDC
prjxray	f4pga	fasm2frames, xc7frames2bit
prjxray-db	zynq7	bitstream database, chipdb for xc7z020

The chip database for the XC7Z020 is generated once from prjxray-db; it is a few hundred megabytes and everything below depends on it.

6.2 Design

[bitstream/blinky/blinky.v](#): a 27-bit counter on the 50 MHz PL oscillator, four LEDs on the top bits, and one line that is not optional:

```
module blinky (input wire sys_clk, output wire [3:0] led);
    reg [26:0] cnt = 27'd0;
    always @(posedge sys_clk) cnt ≤ cnt + 27'd1;
    assign led = cnt[26:23];
    (* keep *) PS7 ps7_i();
endmodule
```

⚠ Rule A Zynq design **must instantiate the PS7 block**, even if it uses nothing of it. nextpnr then ties every unused PS7 input to ground. Without it, the fabric drives the direct nFIQ/nIRQ lines into both Cortex-A9 cores (IRQF2P[19:16], which bypass the interrupt controller) the moment the kernel enables the PL-to-PS level shifters. Both cores spin in FIQ handling forever and nothing is ever printed.

Pins come from [bitstream/blinky/blinky.xdc](#), a plain XDC file that nextpnr reads directly.

6.3 Flow

[bitstream/blinky/Makefile](#) chains four tools and one conversion:

```
blinky.v --yosys synth_xilinx--> blinky.json
        --nextpnr-xilinx----> blinky.fasm
        --fasm2frames-----> blinky.frames
        --xc7frames2bit-----> blinky.bit
        --bit2bin.py-----> blinky.bin
```

```
cd bitstream/blinky && make blinky.bin
```

The last step, [bitstream/bit2bin.py](#), exists because the kernel's zynq-fpga driver expects the layout bootgen produces: the ASCII header of the .bit removed, every 32-bit word byte-swapped.

✓ **Checkpoint** `xxd -l 16 blinky.bin` shows `ffff ffff ffff ffff 6655 99aa`, the sync word in swapped order.

6.4 Load over SSH

The FPGA manager takes a file *name* relative to `/lib/firmware`:

```
ssh root@<ip> 'mkdir -p /lib/firmware; cat > /lib/firmware/blinky.bin' < blinky.bin
ssh root@<ip> 'echo blinky.bin > /sys/class/fpga_manager/fpga0/firmware;
cat /sys/class/fpga_manager/fpga0/state'
```

`tools/stage4-first-load/load-bitstream.sh` does the same with the kernel log redirected to the workstation via netconsole, which is worth having the first time.

✓ **Checkpoint** state reads operating, the four LEDs blink, and the board still answers on port 22.

7 Stage 5: The bitstream API as an image

7.1 The service

`api/app.py` is a FastAPI application with three endpoints:

GET	<code>/state</code>	FPGA manager state and what was loaded last
POST	<code>/bitstream</code>	multipart file=, .bit or .bin, optional ?sha256=; verifies the checksum, finds the sync word, checks the IDCODE packet against the XC7Z020, converts a .bit, loads, answers when the fabric is up
DELETE	<code>/bitstream</code>	loads <code>empty.bin</code> , a PS7-only design, because the FPGA manager has no unload

The IDCODE check looks for the type-1 packet `0x30018001` in the header and compares the following word, masked to 28 bits, with `0x23727093`; the top four bits are the silicon revision. Errors come back as `{"error": "..."} with 400 or 500`. The interactive documentation is served at `/docs`.

7.2 Recipes

All in `yocto/meta-ax7020`, paths relative to it:

`classes/pypi-wheel.bbclass`

installs a pure-Python wheel by unzipping it into site-packages. scarthgap has no class for the pdm-backend that FastAPI's build metadata asks for, and a wheel is already the installed tree.

`recipes-devtools/python/`

fastapi 0.111.1, starlette 0.37.2, uvicorn 0.30.6, click 8.1.7, python-multipart 0.0.9 as wheels; pydantic, anyio and h11 from meta-python.

[recipes-devtools/uv/uv_0.12.13.bb](#)

astral's static armv7-musl binary, hash pinned, for `uv run --with ...` experiments on the board. Not in the service's start path.

[recipes-support/ax7020-api/ax7020-api.bb](#)

`app.py` to `/opt/ax7020-api`, `empty.bin` to `/lib/firmware`, and the unit below, enabled.

[recipes-core/images/ax7020-api-image.bb](#)

core-image-minimal plus the above, Python, dropbear and the operator key. `INITRAMFS_MAXSIZE` raised to 512 MB. **Without** the updater.

[yocto/meta-ax7020/recipes-support/ax7020-api/files/ax7020-api.service](#):

```
[Service]
Type=simple
WorkingDirectory=/opt/ax7020-api
ExecStart=/usr/bin/python3 -m uvicorn app:app --host 0.0.0.0 --port 8000
Restart=on-failure
RestartSec=3
```

⚠ Rule The API image must not contain the updater. With `IMAGE_URL` set it would fetch itself and kexec into itself on every boot, forever.

7.3 Build, publish, and update the maintenance system

```
source yocto/setup-build.sh
bitbake ax7020-api-image virtual/kernel          # API image + new maintenance FIT
tools/publish-image.sh AI-heimdall:/srv/ax7020-images
```

The maintenance FIT with the systemd-based updater has to go into flash once more. From the running Linux, over SSH, no jumper: `tools/flash-fit.sh` uploads, writes through `/dev/mtdblock3`, reads back and compares three MD5s. If U-Boot was rebuilt as well, the same script writes `u-boot.img` to `/dev/mtdblock1`:

```
tools/flash-fit.sh <ip> tftp/fitImage          # mtd3, the FIT
tools/flash-fit.sh <ip> tftp/u-boot.img 1      # mtd1, U-Boot proper
```

✓ Checkpoint All three MD5 lines printed by the script match: local file, upload, and flash read-back.

7.4 The cold boot

Power off. **Unplug the JTAG adapter.** Power on.

⚠ Rule An FT232H left on the JTAG port holds TMS at 3.3 V and, through the protection diodes, keeps the board's 3.3 V rail and the flash chip alive across a power cycle. A flash chip that a Linux session left in 4-byte address mode then never resets, and the boot

ROM, which reads with 3-byte addresses, parks at 0xffffffff28. "Power-cycled" is only true with the adapter unplugged.

✔ **Checkpoint** Within about a minute, with nobody logged in, port 8000 answers on the address the DHCP server holds for ax7020, and a bitstream goes in:

```
IP=$(curl -s http://10.42.0.1:8000/get_all_network_clients \
  | jq -r '.clients[] | select(.hostname=="ax7020" and .active_lease) | .ip')
curl http://$IP:8000/state
curl -F file=@bitstream/blink/blink.bit http://$IP:8000/bitstream
```

```
{"state":"operating", "sha256":"f69a32d2...", "bytes":4045524,
  "idcode":"0x03727093", "filename":"blink.bit"}
```

8 Operating the board

8.1 Finding it

U-Boot, the maintenance kernel and the API kernel each take their own DHCP lease, so the address changes across a boot. Query the DHCP server by hostname rather than remembering an address. The inventory API used above is [dhcp-inventory-api](#); any DHCP server's lease file works the same way.

8.2 SSH

The root filesystem is in RAM, so dropbear generates a new host key on every boot. Before each session:

```
ssh-keygen -R <ip>
```

Connection refused right after a boot means only that dropbear is not up yet; key generation takes about 30 s on this CPU.

8.3 Rolling back an image

On the image server, point [ax7020-latest](#) at an older directory and power-cycle the board. Nothing in flash changes.

8.4 Rewriting flash

[tools/flash-fit.sh](#) for the FIT (mtd3) and U-Boot proper (mtd1), from the running Linux. [boot.bin](#) (mtd0) is written only through the JTAG procedure of Stage 1, and the environment (mtd2) belongs to U-Boot.

8.5 Backing up flash

From U-Boot, over TFTP:

```
tools/ncsh.py <ip> "sf probe 0 30000000 0" "sf read 0x10000000 0 0x2000000" \
```

```
"md5sum 0x10000000 0x20000000" "tftpput 0x10000000 0x20000000 ax7020-qspi.bin"
```

9 Where to go from here

The pipeline is complete as a lab system. Three additions turn it into something that can face a wider network, in this order:

1. **Authentication** on the API. A bitstream with an AXI master has full access to system memory; loading unreviewed bitstreams is remote code execution with extra steps.
2. **A bitstream in the manifest.** A fourth line, `bitstream <file> <sha256>`, loaded by the service at start, so the fabric is never empty after boot.
3. **A build stamp** in `/etc/ax7020-release`, reported by `/state`, so the running image can be told from the one in flash and the one on the server.

The reasoning behind every decision in this document, and the detours that produced the rules in the amber boxes, are in the five articles at maxclerkwell.tech/alinx:

- [Deploying Hardware Like Software: A Bitstream Pipeline for the Zynq](#)
- [ALINX AX7020 Bring-up: Mainline U-Boot Over JTAG, No FSBL, No Serial Cable](#)
- [Stages 2 & 3: A Yocto Linux in QSPI Flash That Fetches Its Own Updates](#)
- [Stages 4 & 5: An Open Bitstream Over REST, and the One Missing Line That Hangs Both Cores](#)
- [Stage 5 Complete: Cold Power-On to a Bitstream API, Nobody Logged In](#)

About the author



Stephan Bökelmann

MaxClerkwell · Engineer, physicist, freelance consultant · Bochum, Germany

Stephan works across the full vertical stack of engineering: silicon-level measurement systems, embedded firmware, PCB design and bring-up, FPGA development, and the monitoring and data infrastructure above them. The common thread is decentralised data acquisition: getting reliable measurements out of physical systems, whether that system is a particle detector, a production line or a development board that has stopped talking. He is a trained mechanic and electrician, a state-certified mechanical engineer, holds a B.Eng. in electrical engineering and an M.Eng. in computer engineering, and is completing a PhD in experimental hadron physics at Ruhr-Universität Bochum, working on HV-MAPS sensors for the PANDA luminosity detector at FAIR. He is managing director of the engineering office [nabla B](#), COO of AUTO INTERN and system architect of its skAlnet Edge-Compute platform, and lectures on programming and databases at THGA Bochum. He is the principal organiser of emBO++, a co-organiser of KiCon Europe, and holds several patents in measurement technology. Professionally active since 2007.

Available for projects

Work like this tutorial is what I do professionally, through my company [nabla B](#) in Bochum: board bring-up without vendor lock-in, embedded Linux with Yocto and mainline U-Boot, FPGA development with open or vendor toolchains, PCB design in KiCad from schematic to EMC, and data-acquisition systems from the sensor to the server. On-site in the DACH region, remote worldwide, in English or German, from feasibility study to serial product.

✉ Commissioned work	office@nabla-b.engineering
✉ Direct	stephan@boekelmann.net
✉ Board vendors and collaborations	collaboration@maxclerkwell.tech
📁 Services and references	maxclerkwell.tech/hire
🌐 LinkedIn	linkedin.com/in/accelerator-stephan

Blog and article series: [maxclerkwell.tech](#) · [maxclerkwell.tech/alinx](#)

Sources for this tutorial: github.com/MaxClerkwell/ax7020-bringup

Discord, Full Stack Engineering: discord.gg/2BXuUY6hrX

© ⓘ This document is licensed under [Creative Commons Attribution 4.0 \(CC BY 4.0\)](#): use, share and adapt it freely, with attribution to Stephan Bökelmann (MaxClerkwell) and a link to [maxclerkwell.tech/alinx](#).

The AX7020 was provided by ALINX as a sponsorship; the vendor had no influence on its content.